

DESIGN SPECIFICATION · ACCOUNTING →
PURCHASING

Three quotes, one memory: purchasing as a system that *learns*.

A sub-module inside HydroNexis-AI Accounting for side-by-side supplier comparison, purchase history pulled from SOS Inventory, and a Nexi-driven RFQ outreach loop that keeps every reply so the next comparison is smarter than the last.

Module · Accounting

View ·

acct_purchasing

Branch · claude/accounting-purchasing-suppliers-5p94sz

Head · 7403d49

Doc rev · 2026-07-27

§ 1

Purpose & scope

What this sub-module is for, and what it deliberately isn't.

Every non-trivial purchase at AteretPaz is meant to be shopped across three suppliers. Today that shopping happens in email threads and paper quotes; nothing carries forward. The Purchasing sub-module is a place to do the comparison once, in one shape, and keep it — so the next time the same product comes up, the answer is already half-written.

It handles: side-by-side supplier comparison (unit / specs / quantity / cost / delivery / other), purchase history per product-supplier pair, a supplier ledger with activity status, and a Nexi-driven outreach flow that dispatches RFQs by email or SMS and logs the reply.

It does not: replace SOS Inventory as the system of record for POs or receipts, replace QuickBooks as the ledger of bills, or approve payments. It is a decision-support and audit layer that reads from those systems and writes intent — quotes requested, quotes received, awards made.

§ 2

Where it lives

Navigation, view registration, discovery.

- › **Primary view** · registered as `acct_purchasing`, dispatched by the app-wide `switchView()` the same way `acct_vendors`, `acct_customers`, `acct_invoices` and `acct_tax` are. Mounted lazily inside `#appMain`.
- › **Module attribution** · added to the `VIEW_TO_MODULE` map with value `accounting`, so the module header banner and permission checks resolve correctly.
- › **Deep-link from Accountant-AI overlay** · a new `purchasing` entry in `ACCT_TABS` shows a summary panel and a “Open Purchasing workspace” button that closes the overlay and calls `switchView('acct_purchasing')`.
- › **Direct URL / console** · `switchView('acct_purchasing')` from anywhere.

§ 3

Four tabs

One header, four working surfaces.

Header banner shows totals: comparisons, open comparisons, products tracked, suppliers used, total logged spend. Underneath, an integration strip: SOS connection state, QB cache state, history-row counts (SOS-imported vs manual), last-sync timestamp, and inline sync controls. Below the header sits the tab strip and the primary action  NEXI · FIND & CONTACT SUPPLIERS .

TAB	WHAT IT DOES	WRITES TO
 Comparisons	Add a product to shop for; capture up to three supplier quotes side by side; expand a card to see the matrix, the cheapest badge, per-supplier history, and a SOS item hint. Actions: award a winner, close, delete, log a purchase from a chosen quote, or ask Nexi for more quotes.	hydroPro_acct_purch_comparisons
 Purchase History	Flat log of every purchase — product, supplier, unit, qty, unit cost, total, PO ref, notes. Filterable search, CSV export. Rows imported from SOS show a SOS badge and cannot be manually deleted.	hydroPro_acct_purch_history
 Suppliers Ledger	Roll-up per supplier: orders count, distinct products, total qty, total spend, last purchase date, activity status (ACTIVE / DORMANT / NEW), recent products chips.	read-only rollup

TAB	WHAT IT DOES	WRITES TO
 Nexi RFQ Log	Every RFQ Nexi dispatched: sent-at, product, supplier, channel, response status, quoted unit cost, lead days, notes. Marking a row <i>quoted</i> auto-appends a soft quote-record to Purchase History so it appears in future comparisons.	<code>hydroPro_acct_purch_outreach</code>

§ 4

Data model

Every persistent shape, in one place.

Everything is stored in `localStorage` under the `hydroPro_*` prefix used across the app. Nothing is written to a backend without going through the existing hnx-sync worker (SOS pulls only).

KEY	SHAPE	PURPOSE
<code>hydroPro_acct_purch_comparisons</code>	array	Saved 3-supplier comparison sheets.
<code>hydroPro_acct_purch_history</code>	array	Merged purchase log — manual entries, SOS-imported PO lines, and RFQ-quote softlines.
<code>hydroPro_acct_purch_outreach</code>	array	RFQ send log with response tracking — the memory Nexi learns from.
<code>hydroPro_acct_purch_sync_meta</code>	object	SOS import status: <code>{status, lastSync, poCount, added, updated}</code> .
<code>hydroPro_acct_purch_rfq_template</code>	string	User-editable RFQ message template with variables.
<code>hydroPro_acct_vendors</code>	array	Existing Accounting Vendors list — Nexi reads it for contact info and writes web-found candidates back.
<code>hydroPro_inv_suppliers_v1</code>	array	Existing Inventory Master suppliers — richer contact data + reliability score.
<code>hydroPro_sos_items_v1</code>	array (IDB)	SOS items cache — read for preferred vendor and last purchase cost per SKU.
<code>hydroPro_emailjs_config_v1</code>	object	Existing EmailJS credentials — reused for silent RFQ auto-send.

Comparison record

```
{
  id: "pc_lqm7...",
  product: "Rockwool cube 100×100",
  category: "Growing media",
  needBy: "2026-08-14",
  requester: "EB",
  status: "open" | "awarded" | "closed",
  chosenSupplier: 0 | 1 | 2 | null,
  createdAt: 1721001600000,
  awardedAt: ...,
  suppliers: [           // up to 3 slots
    { name, unit, qty, specs, unitCost, deliveryDays, payTerms, warranty, notes }
  ]
}
```

Purchase-history record · a single row per PO line or per manual entry.

```
{
  id: "ph_sos_2026-21-P00528-01_0",
  source: "sos" | "manual" | "rfq-quote",
  poNumber: "2026-21-P00528-01",    // sos only
  lineIdx: 0,                       // sos only – dedup key
  date: "2026-05-28",
  product: "Business Card 2×3.5",
  supplier: "Micropoint Graphics",
  sku: "",
  unit: "pc",
  qty: 700,
  receivedQty: 0,
  unitCost: 6.75,
  totalAmount: 4725,
  ref: "2026-21-P00528-01",
  status: "open" | "partial" | "closed",
  notes: "From SOS PO · open",
  importedAt: 1721001600000
}
```

Outreach record · one row per RFQ dispatched (per channel per supplier).


```
{
  id: "rfq_lqzmz..._0",
  sentAt: 1721001600000,
  product, qty, unit, specs, needBy, requester,
  supplier, supplierEmail, supplierPhone,
  subject: "RFQ · Rockwool cube 100×100",
  message: "Hello ...",           // fully-rendered body
  channel: "email" | "sms",
  delivery: "emailjs" | "mailto" | "sms-app" | "error",
  status: "sent" | "opened" | "error",    // dispatch result
  responseStatus: "pending" | "quoted" | "declined" | "no-response",
  responseAt: ...,
  quotedUnitCost: ...,
  quotedDeliveryDays: ...,
  responseNotes: "..."
}
```

§ 5

Comparison logic

How the matrix picks the cheapest and reads the history.

Cheapest badge · for each supplier column, `total = qty × unitCost`. Only columns with a positive total are eligible; the lowest wins the  **CHEAPEST** tag. Ties resolve to first-filled.

Award vs cheapest · the buyer's choice can differ from the cheapest — the column with the buyer's award gets a green top-border and a  **CHOSEN** action; the cheapest tag stays independently. When a comparison has no explicit award yet, the cheapest is used as the visual default.

NEW / EXISTING supplier badge · a supplier is *EXISTING* if their name appears in the merged history OR in the Vendors master (case-insensitive, trimmed). Otherwise *NEW*. This is computed at render time — no separate state.

Per-product history sub-row · `_purchLookupHistory(product, supplier)` filters merged history to matching product-supplier pairs and returns `{times, totalQty, totalCost, lastDate, rows[]}`. The card renders the last six line-items so the buyer sees drift in quantity and price without opening another view.

SOS item hint row · `_purchSosItemHint(product)` scans `window.__hnxSos.getItems()` for a name/sku match and returns the item's preferred vendor, SKU, last purchase cost and on-hand. Rendered as a single blue-tinted row between the criteria table and the history rows. If the supplier under review matches SOS's preferred vendor, an extra note surfaces on that column's history cell.

§ 6

Purchase history sources

Three streams merged into one log; every row is source-tagged.

SOS Inventory · `source: "sos"`

`_purchSyncFromSOS()` calls `/sos/purchase-orders` through the existing hnx-sync Cloudflare worker.

Every PO line becomes one history row. Dedup key is `poNumber + lineIdx`, so re-syncs update in place instead of duplicating.

Trigger: manual (button in header banner). Auto-import on first empty load can be added later.

Manual entry · `source: "manual"`

The History tab has an add form: product, supplier, date, unit, qty, unit cost, PO/ref, notes. Also written by “+ Log purchase” on a comparison card, using the chosen supplier’s quote.

Fully editable and deletable.

RFQ quotes · `source: "rfq-quote"`

When an outreach row is marked *quoted* with a unit price, a soft row is auto-appended: `qty:0, unitCost:<quote>`. It surfaces the price to future comparisons without pretending a purchase happened.

Suppresses double-counting because `qty*unitCost=0`.

QuickBooks · deferred

QB currently caches only trial-balance and aging in `hydroPro_acct_qb_v1`, not per-line vendor-item bills. The banner reads its connection state as a hint.

A `_purchSyncFromQB()` can be added in parallel, using the same `source:` dedup pattern; the rest of the UI needs no change.

GUARANTEE

Once `_purchSyncFromSOS()` has run, every card’s per-supplier “n× before · P× total · last DATE” row is real. The buyer never types history — they type comparisons.

§ 7

Nexi outreach flow

From “find suppliers” to logged RFQ, in five steps.

Entry points: the header primary button, or “📄 Ask Nexi for more quotes” on any comparison card. Both open the outreach modal, prefilled with product / qty / unit / need-by from the card if launched from context.

Step 1

Gather candidates

`_purchGatherAllSuppliers(product)` unions Inventory Master, Accounting Vendors, SOS items’ preferred vendors, past PO vendors, and past-outreach responders.

Step 2

Rank & tag

Product-history match > has email > has phone > reliability score > alpha. Each row shows source chips, `EXISTING` / `NEW` / `HAS HISTORY`, and past reply-rate.

Step 3

User picks

Manual checkboxes, or one-click “top 3” / “existing”. A separate form lets the user add a supplier they found on the web — saved to Vendors master so Nexi remembers.

Step 4

Compose

Editable RFQ template with variable substitution: `{{product}}` `{{qty}}` `{{unit}}` `{{specs}}` `{{needBy}}` `{{supplierContact}}` `{{requester}}` `{{companyName}}`.

Step 5

Send & log

Email via existing EmailJS auto-send (silent) or `mailto:`. SMS via device `sms:` URL scheme with body ≤ 380 chars. Every dispatch appends one row to the outreach log with full body archived.

Channel semantics

CHANNEL	DELIVERY MODE	BEHAVIOR
 EMAIL	emailjs	Silent send via existing EmailJS integration. Status "sent". Requires the shared EmailJS config already used for report auto-send.
	mailto	Fallback when EmailJS is not configured. Opens the device mail client with recipient, subject, and body pre-filled. Status "opened" — the app can't verify actual send.
 SMS	sms-app	Opens the device SMS composer via <code>sms:</code> URL scheme. Body auto-shortened to 380 chars with an ellipsis. Status "opened".
	error	Dispatch threw. Full error stored in the row's <code>error</code> field.

§ 8

Nexi intelligence

The learning loop that makes tomorrow’s comparison start with an answer.

The purchasing sub-module registers a Nexi domain via `registerNexiDomain()`, using the app-wide domain registry pattern (Irrigation, Logistics, etc. all register the same way). The registration exposes the three collections and provides a custom `answer(qLower)` function.

ASK NEXI	WHAT SHE READS	WHAT SHE RETURNS
<i>“who supplies rockwool?”</i>	All supplier sources + history	Ranked list with contact info + past-order counts
<i>“best price on rockwool cubes”</i>	Purchase history (incl. RFQ quotes)	Cheapest supplier + next two, with reply-rate
<i>“supplier reply rate”</i>	Outreach log	Per-supplier quoted/sent scoreboard
<i>“how many RFQs have I sent”</i>	Outreach log	Totals + open comparisons + records

THE COMPOUNDING EFFECT

Every outreach is logged. Every quote reply is logged. Every awarded winner is logged. On the third comparison for the same product, Nexi’s ranking already knows: who quoted, who replied within a day, who was cheapest, who won last time. Manual entry is only ever for what hasn’t happened yet.

§ 9

Permissions & audit

Who sees, who edits, what's recorded.

- › The Accounting role has full `manageHR` and payroll access and read-only access to operational modules — the Purchasing sub-tab sits within Accounting and inherits the module's permission gate.
- › Each outreach row records `requester` from the current user profile (`getCurrentUser()`). Once an RFQ has been dispatched, its body and dispatch metadata are immutable; only the response fields (`responseStatus`, `quotedUnitCost`, `quotedDeliveryDays`, `responseNotes`) are editable.
- › SOS-imported history rows are protected from manual deletion — `_purchDeleteHistory()` explicitly refuses them and tells the user to delete the PO in SOS or use "Clear SOS-imported rows".
- › CSV export writes `source`, `status`, `ref`, and `receivedQty` alongside the numeric fields, so an offline audit can trace every row to its origin.

§ 10

Wiring points

Exactly which globals the sub-module touches.

GLOBAL / HOOK	READ?	WRITTEN?	PURPOSE
<code>switchView(v)</code>	hooked	—	Registers <code>acct_purchasing</code> in the dispatcher.
<code>VIEW_TO_MODULE</code>	—	merge	Adds <code>acct_purchasing: "accounting"</code> .
<code>ACCT_TABS</code>	—	splice	Adds the <code>purchasing</code> sub-tab to the Accountant-AI overlay dropdown.
<code>window.__hnxApi(...)</code>	call	—	Fetches <code>/sos/purchase-orders</code> for history import.
<code>window.__hnxSos.getItems()</code>	call	—	Reads cached SOS items for the item hint row.
<code>window.__hnxSos.getMeta()</code>	call	—	SOS connection state for the banner.
<code>_sendViaEmailJs(to, subj, body)</code>	call	—	Silent RFQ auto-send via the existing EmailJS integration.
<code>registerNexiDomain({...})</code>	call	—	Exposes the purchasing intelligence to Nexi's chat/brain.
<code>showToast(msg, tone)</code>	call	—	User feedback on sync / send / save.
<code>getCurrentUser()</code>	call	—	Requester name in the outreach signature.

§ 11

Limits & roadmap

What today's build deliberately doesn't do — and what unlocks each capability.

NOT YET	WHY	WHAT UNLOCKS IT
Automated web supplier search	A browser HTML app can't crawl external sites (CORS, no server).	Add <code>/nexi/supplier-search?q=</code> to the hnx-sync worker; the modal already has a slot for it.
Silent SMS auto-send	Browsers have no direct SMS API; <code>sms</code> : hands off to the device app.	Add a Twilio-backed <code>/nexi/sms</code> worker route; swap the <code>sms</code> : handoff for a POST.
QB bill-line import	Current QB cache is TB/aging only.	Add a Make scenario for <i>Bills by Vendor with Lines</i> ; feed <code>_purchSyncFromQB()</code> .
Automatic reply detection	Response state is human-set today.	An inbound-email webhook (Zapier/Make on your quotes inbox) → <code>/nexi/rfq-reply?rfqId=</code> → auto-flip <code>responseStatus</code> .
Multi-currency & VAT split	Amounts are single-currency, VAT-inclusive.	Add <code>currency</code> , <code>vatRate</code> , <code>vatMode</code> to history rows and quote fields; render alt-currency next to totals.
Approval workflow > PX	Awards are direct today.	Add a threshold rule + approver assignment on the comparison card, gated by role.
Purchase Order creation back into SOS	Only pulls from SOS, doesn't push.	SOS <code>/sos/purchase-orders</code> POST via a new worker route on award.

Nothing on this list breaks the current model. Each item extends a stored field, adds one worker route, or plugs a new record source into the same merged history log.

HydroNexis-AI · Accounting · Purchasing sub-module

Design revision 2026-07-27 · branch `claude/accounting-purchasing-suppliers-5p94sz`